Department of Computer Science and Engineering

CSA06 – Design and Analysis of Algorithms

Assessment	CO2 – Debugging and Optimisation Task (SOC Alert-Triage Module of "SentinelOps")
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	50 (Assessment Tool 2: 100 points on Google Classroom)
Mode	Individual
Student Reg. No.	192511124
Student Name	Deepak S
GitHub repository	https://github.com/<your-username>/sentinelops-co2 (replace with your actual link)

Approach. The code was debugged by reasoning and testing, not rewritten: every fix is a small edit to the original line, marked FIX 1 – FIX 9 in the source. All outputs below were produced by actually running the programs (Python 3). Where a figure depends on the original program's indentation (lost in the question paper), the assumption is stated.

(a) Reproduce and Identify the Defects [10 marks]

1. Test data

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in km
```

2. Original program run on the test data

original.py (source)

```
import math

def sort_alerts(alerts):  # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):  # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):  # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Screenshot 1 – Original (defective) source code

```
$ python3 original.py

sort_alerts      : [('A4', 9), ('A3', 5), ('A3', 5), ('A5', 1), ('A5', 1)]
is_blocked('10.0.0.9'  ): False
is_blocked('10.0.0.5'  ): False
is_blocked('8.8.8.8'   ): False
find_signature   : [0, 4]
closest_logins   : ((0, 2), 60.440052945046304)
```

Screenshot 2 – Execution of the original program

Assumption: in is_blocked the question paper lost its indentation; the “else” is read as belonging to the “if” inside the loop (if match … else …), which is what makes blocked addresses “get through”.

3. Expected vs actual output

Function	Expected output	Actual output (original)	Observation
sort_alerts	`[('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]`	`[('A4', 9), ('A3', 5), ('A3', 5), ('A5', 1), ('A5', 1)]`	A1 and A2 vanished; A3 and A5 duplicated

is_blocked (3 queries)	`10.0.0.9 → True`
	`10.0.0.5 → True`
	`8.8.8.8 → False`
		`10.0.0.9 → False`
		`10.0.0.5 → False`
		`8.8.8.8 → False`
			Blocked IPs are never reported as blocked

find_signature	`[0, 8]`	`[0, 4]`	Signature at the end (index 8) missed; false hit at index 4 (“FAIX”)
closest_logins	`((0, 5), 1.4142…)` `locations (2,3) and (3,4)`	`((0, 2), 60.4401)`	Reports the FARTHEST pair, (2,3) and (40,50), 60.44 km apart

4. Defects located (nine)

#	Function	Faulty line	Nature of the error
1	sort_alerts	`while j > 0 and alerts[j][1] < key[1]:`	Loop condition off by one: index 0 is never compared or shifted (logic / boundary error).
2	sort_alerts	`alerts[j] = key`	Key is inserted one slot too far left; it must go to position j + 1 (indexing error).
3	is_blocked	`return False  (inside if blocklist[i] == ip)`	Wrong return value on a successful match; must be True.
4	is_blocked	`else: return False`	Premature exit: gives up after the first element; also no final return, so an empty list returns None.
5	find_signature	`for i in range(n - m):`	Loop bound off by one: the last window (i = n – m) is never examined.
6	find_signature	`if j == m - 1:`	Tested after the inner loop even when it ended by break at the last character → false hits.
7	closest_logins	`min_dist = 0`	Wrong initial value; for a minimum it must be +infinity.
8	closest_logins	`for j in range(len(events)):`	Pairs a point with itself (distance 0) and examines every pair twice; j should start at i + 1.
9	closest_logins	`if d > min_dist:`	Comparison reversed: finds the maximum, not the minimum.

(b) Root Causes and Systematic Debugging [12 marks]

1. Root cause of each defect

#	Root cause	How it produces the observed wrong output
1	The inner loop guard uses j > 0 instead of j >= 0, so A[0] is treated as a sentinel that never takes part.	An element that belongs in front of A[0] (every new maximum) can never move there: A2 (9) is not placed ahead of A1 (3).
2	After the while-loop j points at the element that is NOT smaller than key (or at −1). The gap is j + 1.	Key overwrites A[j], destroying that alert (A1/A2 disappear) while the shifted copy stays (A3/A5 duplicated). With j = −1 it even writes to the last slot.
3	Return values swapped (True/False confused).	A match is reported as “not blocked”.
4	The else belongs to every loop iteration instead of to the loop as a whole; “not equal to this entry” was mistaken for “not in the list”.	Search stops after entry 0. Together with defect 3 the function can never return True.
5	The number of alignments of a length-m pattern in a length-n text is n − m + 1, not n − m (fence-post error).	Window i = 8 (“FAIL” at the end of the log) is never tried → true signature missed. Equally, when n = m nothing is scanned.
6	Match detection relies on the loop variable j after a break instead of on “no break occurred”.	At i = 4 the window “FAIX” breaks at j = 3 = m − 1, so the test j == m − 1 is still true → false signature reported.
7	min_dist starts at 0, so no real distance can ever be smaller than it.	With the correct comparison (<) no pair would ever be recorded; result would stay (None, 0).
8	Nested loop ignores symmetry: d(i,j) = d(j,i) and d(i,i) = 0.	Half the work is repeated and, for a minimum search, the self pair (distance 0) would always win.
9	The update rule is “larger than the current best” (maximum search).	With min_dist = 0 it keeps replacing the pair by farther ones and ends with the farthest pair (60.44 km).

2. Pseudocode of the corrected algorithms (written before implementation)

2.1 sort_alerts – corrected insertion sort (descending, stable)

```
ALGORITHM SortAlerts(A[0..n-1])          // A[i] = (id, severity)
  FOR i ← 1 TO n-1 DO
      key ← A[i];  j ← i - 1
      WHILE j ≥ 0 AND A[j].severity < key.severity DO   // strict < keeps equal severities in
arrival order
          A[j+1] ← A[j];  j ← j - 1
      A[j+1] ← key
  RETURN A
```

2.2 is_blocked – corrected sequential search

```
ALGORITHM IsBlocked(L[0..b-1], ip)
  FOR i ← 0 TO b-1 DO
      IF L[i] = ip THEN RETURN True
  RETURN False                              // reached only after ALL b entries were compared
```

2.3 find_signature – corrected brute-force string matching

```
ALGORITHM FindSignature(T[0..n-1], P[0..m-1])
  IF m = 0 OR m > n THEN RETURN empty list
  hits ← empty list
  FOR i ← 0 TO n-m DO                       // n-m+1 alignments
```

```
      j ← 0
      WHILE j < m AND T[i+j] = P[j] DO  j ← j + 1
      IF j = m THEN append i to hits        // all m characters matched
  RETURN hits
```

2.4 closest_logins – corrected brute-force closest pair

```
ALGORITHM ClosestLogins(E[0..n-1])         // E[k] = (x, y)
  IF n < 2 THEN RETURN (None, None)
  best ← +∞;  pair ← None
  FOR i ← 0 TO n-2 DO
      FOR j ← i+1 TO n-1 DO                 // each unordered pair once
          d ← sqrt((xi-xj)² + (yi-yj)²)
          IF d < best THEN best ← d;  pair ← (i, j)
  RETURN (pair, best)
```

3. Step-by-step traces

3.1 Insertion trace for the alerts list (original defective code)

```
start      : ['A1:3', 'A2:9', 'A3:5', 'A4:9', 'A5:1']
i=1 key=A2:9 -> ['A2:9', 'A2:9', 'A3:5', 'A4:9', 'A5:1']
i=2 key=A3:5 -> ['A2:9', 'A3:5', 'A3:5', 'A4:9', 'A5:1']
i=3 key=A4:9 -> ['A4:9', 'A3:5', 'A3:5', 'A3:5', 'A5:1']
i=4 key=A5:1 -> ['A4:9', 'A3:5', 'A3:5', 'A5:1', 'A5:1']
```

i = 1: j = 0 so the loop (j > 0) is skipped and alerts[0] = A2 overwrites A1 – A1 disappears and A2 is never moved ahead of it. The final list contains duplicates.

3.2 Insertion trace (corrected code)

```
start      : ['A1:3', 'A2:9', 'A3:5', 'A4:9', 'A5:1']
i=1 key=A2:9 -> ['A2:9', 'A1:3', 'A3:5', 'A4:9', 'A5:1']
i=2 key=A3:5 -> ['A2:9', 'A3:5', 'A1:3', 'A4:9', 'A5:1']
i=3 key=A4:9 -> ['A2:9', 'A4:9', 'A3:5', 'A1:3', 'A5:1']
i=4 key=A5:1 -> ['A2:9', 'A4:9', 'A3:5', 'A1:3', 'A5:1']
```

A4 (9) is not moved ahead of A2 (9) because 9 < 9 is false – equal severities keep arrival order, as required.

3.3 Alignment table for the log (corrected)

Log = F A I L F A I X F A I L (indices 0–11), pattern = FAIL (m = 4), so there are n − m + 1 = 9 alignments.

i	Window log[i..i+3]	Comparisons	Result (corrected)	Original code
0	FAIL	4	MATCH → append 0	match
1	AILF	1	mismatch	mismatch
2	ILFA	1	mismatch	mismatch
3	LFAI	1	mismatch	mismatch
4	FAIX	4	mismatch	FALSE HIT
5	AIXF	1	mismatch	mismatch
6	IXFA	1	mismatch	mismatch
7	XFAI	1	mismatch	mismatch
8	FAIL	4	MATCH → append 8	never examined

Total character comparisons in the corrected scan: 18. The original reported [0, 4] – index 4 is the false hit (mismatch at the last character), index 8 was never examined.

3.4 Closest-pair trace (given data)

Pairs examined by the corrected brute force (distances in km): (0,1) 28.8 · (0,2) 60.4 · (0,3) 3.6 · (0,4) 12.2 · (0,5) 1.414 ← best · (1,2) 34.4 · (1,3) 29.8 · (1,4) 20.0 · (1,5) 27.5 · (2,3) 60.2 · (2,4) 48.8 · (2,5)

59.0 · (3,4) 11.4 · (3,5) 3.6 · (4,5) 10.8. Smallest = 1.414 km between events 0 and 5, i.e. (2,3) and (3,4).

4. Re-test after every correction

Each fix was applied individually and in combination and the function re-run on the test data. This shows that every defect was independently real and that only the complete set of fixes gives the expected answer.

sort_alerts

Version	Output
original	`[('A4', 9), ('A3', 5), ('A3', 5), ('A5', 1), ('A5', 1)]`
FIX 1 only	`[('A1', 3), ('A1', 3), ('A1', 3), ('A1', 3), ('A4', 9)]`
FIX 2 only	`[('A1', 3), ('A2', 9), ('A4', 9), ('A3', 5), ('A5', 1)]`
FIX 1 + FIX 2	`[('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]`

Expected: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

is_blocked (queries 10.0.0.9, 10.0.0.5, 8.8.8.8)

Version	Output
original	`False, False, False`
FIX 3 only	`False, True, False`
FIX 4 only	`False, False, False`
FIX 3 + FIX 4	`True, True, False`

Expected: True, True, False

find_signature

Version	Output
original	`[0, 4]`
FIX 5 only	`[0, 4, 8]`
FIX 6 only	`[0]`
FIX 5 + FIX 6	`[0, 8]`

Expected: [0, 8]

closest_logins

Version	Output (pair, distance)
original (min=0, j from 0, >)	`((0, 2), 60.4401)`
after FIX 9 only (min=0, j from 0, <)	`(None, 0.0000)`
after FIX 8 (+ j from i+1, min=0, >)	`((0, 2), 60.4401)`
after FIX 7 (min=inf, j from 0, <)	`((0, 0), 0.0000)`
after FIX 7+8+9	`((0, 5), 1.4142)`

Expected: ((0, 5), 1.4142). Fixing only the comparison (FIX 9) gives (None, 0) because min_dist = 0 can never be beaten – this proves defect 7 is genuine; only FIX 7 + 8 + 9 together produce the right pair.

5. Corrected program and its execution

corrected.py (source, FIX comments)

```
import math

def sort_alerts(alerts):  # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:      # FIX 1: j > 0  ->  j >= 0
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key                          # FIX 2: alerts[j]  ->  alerts[j + 1]
    return alerts

def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return True                              # FIX 3: False  ->  True
        # FIX 4: removed the 'else: return False' that quit after the first element
    return False                                     # FIX 4: reached only after ALL entries were checked

def find_signature(log, pattern):  # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    if m == 0 or m > n:                              # boundary guard: nothing to find
        return []
    hits = []
    for i in range(n - m + 1):                       # FIX 5: n - m  ->  n - m + 1
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:                                        # FIX 6: 'if j == m-1' also fired after a break
            hits.append(i)
    return hits

def closest_logins(events):  # brute-force closest pair of login locations
    min_dist = float("inf")                          # FIX 7: 0  ->  infinity
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):          # FIX 8: range(len) -> range(i + 1, len)
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < min_dist:                         # FIX 9: '>'  ->  '<'
                min_dist = d
                pair = (i, j)
    if pair is None:                                 # fewer than two events
        return None, None
    return pair, min_dist
```

Screenshot 3 – Corrected source code (changed lines highlighted)

corrected.py (execution)

```
$ python3 test_corrected.py
sort_alerts    : [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]
is_blocked('10.0.0.9'): True
is_blocked('10.0.0.5'): True
is_blocked('8.8.8.8'): False
find_signature : [0, 8]
closest_logins : ((0, 5), 1.4142135623730951)
```

Screenshot 4 – Corrected program on the test data (all four outputs now as expected)

6. Boundary tests

Function	Boundary case	Corrected output
sort_alerts	empty list []	`[]`
sort_alerts	single alert	`[('X', 4)]`
sort_alerts	equal severities a5,b5,c5	`[('a', 5), ('b', 5), ('c', 5)]`
sort_alerts	smallest first [a1,b2]	`[('b', 2), ('a', 1)]`
is_blocked	empty blocklist	`False`
is_blocked	IP is last entry	`True`
find_signature	pattern at very end ('xxxxFAIL')	`[4]`

find_signature	pattern longer than log	`[]`
find_signature	mismatch on last char ('FAIX')	`[]`
find_signature	overlapping ('aaaa','aa')	`[0, 1, 2]`
closest_logins	empty list	`(None, None)`
closest_logins	one event	`(None, None)`
closest_logins	two events (0,0),(3,4)	`((0, 1), 5.0)`

For comparison, the original program on selected boundary cases:

Function	**Case**	**Original output**
sort_alerts	[('a',1),('b',2)]	`[('b', 2), ('b', 2)]`
is_blocked	empty blocklist	`None`
find_signature	'xxxxFAIL','FAIL'	`[]`
closest_logins	one event	`(None, 0)`
find_signature	pattern longer than log	`[]`
find_signature	'FAIX','FAIL'	`[]`

Notes: the original returns (None, 0) for one event – a fake distance of 0 for a pair that does not exist – and None instead of False for an empty blocklist. The corrected version returns (None, None) and False. A pattern longer than the log, an empty pattern, and a pattern at the very end are all handled by the m = 0 / m > n guard and the n − m + 1 bound.

(c) Optimise the Corrected Module [10 marks]

Every optimisation below is output-preserving; this is verified by the test harness (including 300 random point sets, with many ties, that compare the optimised closest_logins with the brute force).

1. sort_alerts – is insertion sort still suitable?

- **For nearly sorted alerts, yes:** insertion sort is adaptive, running in $\Theta(n + I)$ where I is the number of inversions; with few misplaced alerts it makes about $n - 1$ comparisons (measured: 999 for n = 1,000).
- **For production, no:** nothing guarantees that the 50,000 daily alerts arrive nearly sorted. A random or reverse-ordered burst costs about $n^2/4$ to $n^2/2$ comparisons – roughly 6×10^8 to 1.25×10^9 for n = 50,000. It is also stable and in place, but the quadratic worst case is the problem.
- **Better approach:** Timsort (Python's built-in sorted/list.sort): stable, $\Theta(n)$ on already or nearly sorted input (it detects existing runs), and $O(n \log n)$ always. sort(key=severity, reverse=True) keeps equal severities in arrival order because reverse=True preserves stability. For a small integer severity range (say 1–10), a bucket/counting sort in $O(n + k)$ would also work.

2. is_blocked – repeated lookups

A list scan costs up to b comparisons per query, so q queries cost $O(q \cdot b)$. Build a hash set once with build_blocklist ($O(b)$ time, $O(b)$ memory) and test membership with ip in blocklist: $O(1)$ on average per query, total $O(b + q)$. For 50,000 alerts against a 2,000-entry list this removes about 50 million comparisons (measured: 0.71 s → 0.002 s for 20,000 queries).

3. find_signature – no redundant comparisons

- Window loop stops at $n - m$ (inclusive): nothing is skipped and no window runs past the end of the log.
- The inner loop breaks at the first mismatching character, so a window never compares characters after it has already failed.
- Match detection uses the loop's else clause (no break all m characters matched), which removes the extra "j == m − 1" test executed for every window (8 redundant tests on the test log) and the false hits.
- Immediate return [] for an empty pattern or a pattern longer than the log, so no pointless scan is started.

4. closest_logins – pairs, square roots and pruning

- **Self and repeated pairs eliminated:** inner loop starts at i + 1 ($n(n - 1)/2$ pairs instead of n^2).
- **Square roots removed:** sqrt is monotonic, so squared distances are compared; one sqrt is taken for the final answer only.
- **Pruning:** points are sorted by x once. For a point at position a, scanning right, if $(x_b - x_a)^2$ already exceeds the best squared distance found, every later point is even farther in x and cannot be closer, so the inner loop breaks. The test uses ">" rather than "≥" so ties are still examined and the same pair as the brute force is reported (the lexicographically first (i, j)).

Pseudocode of the optimised algorithms

```
ALGORITHM SortAlertsFast(A)
  STABLE_SORT(A, key = severity, order = descending)     // Timsort
  RETURN A

ALGORITHM BuildBlocklist(L):  S ← empty hash set;  FOR x IN L: insert x into S;  RETURN S
ALGORITHM IsBlockedFast(S, ip):  RETURN (ip IN S)

ALGORITHM FindSignatureFast(T, P)          // n-m+1 windows, break on first mismatch
  IF m = 0 OR m > n THEN RETURN []
  FOR i ← 0 TO n-m DO
      FOR j ← 0 TO m-1 DO
          IF T[i+j] ≠ P[j] THEN BREAK
      ELSE-OF-LOOP (no break): append i
  RETURN hits

ALGORITHM ClosestLoginsFast(E)
  IF n < 2 THEN RETURN (None, None)
  order ← indices of E sorted by (x, index);  xs, ys ← coordinates in that order
  best ← +∞;  bestPair ← None
  FOR a ← 0 TO n-1 DO
      FOR b ← a+1 TO n-1 DO
          dx ← xs[b] - xs[a]
          IF dx² > best THEN BREAK                       // prune
          d2 ← dx² + (ys[b] - ys[a])²
          IF d2 < best OR (d2 = best AND (orig idx pair) < bestPair) THEN update
  RETURN (bestPair, sqrt(best))
```

The complete optimised source code is given in part (e).

(d) Analyse Before and After [10 marks]

1. Operation counts and time complexity

Function	Before (corrected / original)	After (optimised)
sort_alerts	Key comparisons: best n − 1 (sorted), average ≈ $n^2/4$, worst $n(n-1)/2$ (reverse order). Shifts ≈ the same. $O(n^2)$; $\Omega(n)$.	Timsort: best n − 1, worst ≈ $n \log_2 n$ comparisons. $O(n \log n)$; $\Omega(n)$. Example n = 50,000: ≈ 6.25×10^8 (average) → ≈ 7.8×10^5.
is_blocked	Per query: 1 … b comparisons (hit at position k costs k; miss costs b). q queries: $O(q \cdot b)$.	Build once: b hash insertions, O(b). Per query: 1 hash + 1 equality test, O(1) average. Total O(b + q). (Worst case per lookup is O(b) with hash collisions, which is unrealistic.)
find_signature	Original: (n − m) windows, each ≤ m character comparisons plus 1 extra "j == m − 1" test. Worst $(n-m) \cdot m$, best n − m. $\Theta(nm)$ worst – and wrong.	(n − m + 1) windows, ≤ m comparisons each, no extra tests. Worst $(n-m+1) \cdot m$, best n − m + 1. Still $\Theta(nm)$ worst – the brute-force bound – but with fewer operations and the correct answer.
closest_logins	Original: n^2 distance computations, each with a sqrt. Corrected brute force: $n(n-1)/2$ computations, each with a sqrt. $\Theta(n^2)$.	Sort $O(n \log n)$ + pruned scan: $\Theta(n)$ distance computations expected on spread-out data, $O(n^2)$ worst case (all points in one vertical line). Only 1 sqrt in total.

Worked example on the given data. closest_logins with n = 6: original 36 distance computations → corrected brute force 15 → optimised 2. find_signature: original 14 character comparisons + 8 extra tests on 8 windows (result wrong) → corrected 18 comparisons on 9 windows (result right; the 4 additional comparisons examine the previously missed final window) and no extra tests.

2. Measured distance computations for closest_logins

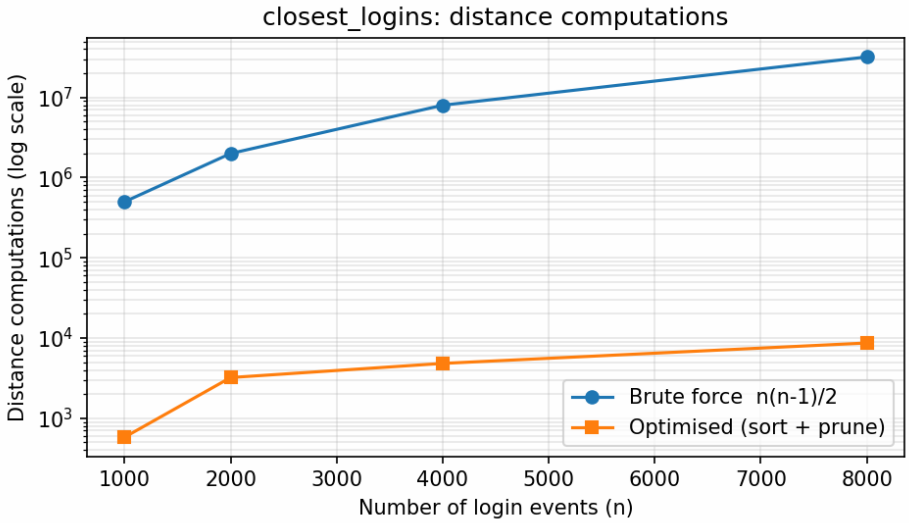
n random login locations were generated uniformly in a 1000 km × 1000 km square (fixed seeds). The brute-force and optimised versions returned the identical pair and distance for every size. Distance computations were counted inside the loops.

Batch size n	Original n^2	Brute force $n(n-1)/2$ (measured)	Optimised (measured)	Reduction vs brute force
1,000	1,000,000	499,500	579	863×
2,000	4,000,000	1,999,000	3,214	622×
4,000	16,000,000	7,998,000	4,825	1658×
8,000	64,000,000	31,996,000	8,674	3689×

Running time on the same inputs (machine-dependent, indicative only):

n	Brute force with sqrt per pair (s)	Optimised (s)	Speed-up
1,000	0.069	0.0006	114×
2,000	0.328	0.0014	229×

4,000	1.456	0.0022	666×
8,000	5.190	0.0066	788×



Graph 1 – Distance computations vs batch size (log scale)

```
experiments.py  (execution)

$ python3 experiments.py

CLOSEST-PAIR  (distance computations, uniform random points)
     n  original(n*n)  brute n(n-1)/2  optimised
  1000      1,000,000         499,500        579
  2000      4,000,000       1,999,000      3,214
  4000     16,000,000       7,998,000      4,825
  8000     64,000,000      31,996,000      8,674

INSERTION SORT  (key comparisons)
     n     random  nearly sorted
  1000    248,174            999
  2000  1,019,643          2,000
  4000  4,032,674          4,001
```

Screenshot 5 – Experiment output

Worst case, for honesty: when all 2,000 events share the same x-coordinate, the x-gap never exceeds the best distance, no pruning is possible, and the optimised version also performs 1,999,000 computations – exactly the brute-force n(n − 1)/2. The pruning therefore improves typical, spread-out data but not the worst-case order.

3. Insertion sort: nearly sorted vs random alerts

Alerts had random severities 1–100. “Nearly sorted” means already in highest-first order with 1 % of adjacent pairs swapped. Comparisons are counted for every evaluation of the test alerts[j][1] < key[1]. Timsort’s comparison count is shown for reference.

n	Insertion: random	Theory n(n−1)/4	Insertion: nearly sorted	Theory ≈ n − 1	Timsort: random	Timsort: nearly sorted
1,000	248,174	249,750	999	999	8,620	999
2,000	1,019,643	999,500	2,000	1,999	18,740	2,036

4,000	4,032,674	3,999,000	4,001	3,999	39,488	4,076



Graph 2 – Sorting comparisons vs number of alerts (log scale)

4. Do the measurements agree with the analysis?

- Brute-force distance counts equal n(n − 1)/2 exactly (499,500; 1,999,000; 7,998,000; 31,996,000) and quadruple when n doubles – Θ(n²) confirmed. The original's n² is twice that.
- The optimised count grows roughly linearly (579 → 3,214 → 4,825 → 8,674 for n = 1,000 → 8,000), about 1 computation per event instead of n/2 – a reduction of about 3,700× at n = 8,000. This matches the expected Θ(n) on spread-out data (the exact figures fluctuate with where the closest pair happens to be). The worst-case test confirms the O(n²) bound is unchanged for degenerate data.
- Insertion sort on random alerts costs 248,174 / 1,019,643 / 4,032,674 comparisons, within 1–2 % of n(n − 1)/4 and four times larger each time n doubles – quadratic as derived. On nearly sorted alerts it costs 999 / 2,000 / 4,001, i.e. ≈ n – linear, again as predicted.
- Timsort needs ≈ 8.6×10^3 to 3.9×10^4 comparisons on random alerts (≈ n log n, about 100× fewer than insertion sort at n = 4,000) and ≈ n on nearly sorted input, so it is never worse than insertion sort asymptotically and is far better in the worst case.
- The blocklist set lookup is 350× faster than the list scan on the test workload (0.71 s vs 0.002 s), as O(q·b) → O(b + q) predicts.

Small differences (e.g. 2,000 instead of 1,999) come from the random placement of the swapped pairs; wall-clock times vary between machines, but the operation counts are deterministic and agree with the derivations.

(e) Production-Quality Code [8 marks]

Final optimised module (sentinelops.py): docstrings on every function, comments for each optimisation, and a test harness (run_tests) that checks every function including the boundary cases from (b). Run with: python sentinelops.py

```
"""SentinelOps SOC alert-triage module (debugged and optimised).

Functions
---------
sort_alerts       : order alerts by severity, highest first, stable.
build_blocklist   : one-off preprocessing of the blocklist into a hash set.
is_blocked        : membership test for a source IP address.
find_signature    : every start index of an attack signature in a log line.
closest_logins    : closest pair of login locations (index pair, distance).

Every function returns exactly what the corrected brute-force version returns;
only the amount of work changes.  Run this file to execute the test harness.
"""
import math
import random


# ---------------------------------------------------------------------------
# 1. Sorting
# ---------------------------------------------------------------------------
def sort_alerts(alerts):
    """Sort (alert_id, severity) tuples by severity, highest first.

    Equal severities keep their arrival order (stable).  Python's Timsort is
    used: O(n) on nearly sorted input, O(n log n) in the worst case, and
    ``reverse=True`` preserves stability.  The list is sorted in place and
    returned, like the original function.
    """
    alerts.sort(key=lambda a: a[1], reverse=True)
    return alerts



def insertion_sort_alerts(alerts):
    """Corrected insertion sort (kept for comparison / tiny inputs)."""
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:   # strict '<' => stable
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts



# ---------------------------------------------------------------------------
# 2. Blocklist lookup
# ---------------------------------------------------------------------------
def build_blocklist(ips):
    """Build the hash-set index once: O(b)."""
    return frozenset(ips)



def is_blocked(blocklist, ip):
    """Return True if ``ip`` is on the blocklist.

    ``blocklist`` may be a list (sequential scan, O(b)) or the set returned by
    build_blocklist (hash lookup, O(1) on average).  An empty blocklist
    blocks nothing.
    """
```

```
    return ip in blocklist



# ---------------------------------------------------------------------------
# 3. Signature scan (brute-force string matching)
# ---------------------------------------------------------------------------
def find_signature(log, pattern):
    """Return every start index at which ``pattern`` occurs in ``log``.

    Windows run from 0 to n-m inclusive (no window is missed, none is invented
    past the end).  The inner loop stops at the first mismatch and the match
    test is the loop's ``else`` clause, so no comparison is repeated or wasted.
    An empty pattern, or one longer than the log, yields [].
    """
    n, m = len(log), len(pattern)
    if m == 0 or m > n:
        return []
    hits = []
    for i in range(n - m + 1):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:                       # no break => all m characters matched
            hits.append(i)
    return hits



# ---------------------------------------------------------------------------
# 4. Closest pair of login locations
# ---------------------------------------------------------------------------
def closest_logins_bruteforce(events, stats=None):
    """Reference brute force: every unordered pair once, sqrt only for the answer.

    Returns ((i, j), distance) with i < j, or (None, None) for < 2 events.
    Ties are resolved in favour of the lexicographically first (i, j).
    If ``stats`` is a dict, stats['dist'] receives the number of distance
    computations.
    """
    n = len(events)
    best, pair, count = float("inf"), None, 0
    for i in range(n):
        xi, yi = events[i]
        for j in range(i + 1, n):
            dx, dy = xi - events[j][0], yi - events[j][1]
            d2 = dx * dx + dy * dy          # squared distance: no sqrt needed
            count += 1
            if d2 < best:
                best, pair = d2, (i, j)
    if stats is not None:
        stats["dist"] = count
    return (None, None) if pair is None else (pair, math.sqrt(best))



def closest_logins(events, stats=None):
    """Closest pair of (x, y) login locations, with pruning.

    * Points are sorted by x once (O(n log n)).
    * Each point is compared only with points to its right (no self pairs, no
      repeated pairs).
    * Squared distances are compared; a single sqrt is taken at the end.
    * Pruning: once the x-gap alone squared exceeds the best squared distance,
      no later point (even further right) can be closer, so the inner loop
      stops.  The gap test uses '>' (not '>=') so equal-distance ties are still
      examined and the same pair as the brute force is reported.

    Returns ((i, j), distance) with i < j (original indices), or (None, None)
    when there are fewer than two events.
```

```
    """
    n = len(events)
    if n < 2:
        return None, None
    order = sorted(range(n), key=lambda k: (events[k][0], k))
    xs = [events[k][0] for k in order]
    ys = [events[k][1] for k in order]
    best, best_pair, count = float("inf"), None, 0
    for a in range(n):
        xa, ya = xs[a], ys[a]
        for b in range(a + 1, n):
            dx = xs[b] - xa
            if dx * dx > best:              # prune: remaining points are farther
                break
            dy = ys[b] - ya
            d2 = dx * dx + dy * dy
            count += 1
            if d2 < best:
                best = d2
                best_pair = (min(order[a], order[b]), max(order[a], order[b]))
            elif d2 == best:                # tie: keep brute-force's first pair
                cand = (min(order[a], order[b]), max(order[a], order[b]))
                if cand < best_pair:
                    best_pair = cand
    if stats is not None:
        stats["dist"] = count
    return best_pair, math.sqrt(best)



# ---------------------------------------------------------------------------
# Test harness
# ---------------------------------------------------------------------------
def _check(name, got, expected):
    status = "PASS" if got == expected else "FAIL"
    print(f"[{status}] {name}: got {got!r}")
    assert got == expected, f"{name}: expected {expected!r}"



def run_tests():
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    log, pattern = "FAILFAIXFAIL", "FAIL"
    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    expected_sorted = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]

    print("--- sort_alerts ---")
    _check("given data (fast)", sort_alerts(list(alerts)), expected_sorted)
    _check("given data (insertion)", insertion_sort_alerts(list(alerts)), expected_sorted)
    _check("empty list", sort_alerts([]), [])
    _check("single alert", sort_alerts([("X", 4)]), [("X", 4)])
    _check("all equal severity keeps arrival order",
           sort_alerts([("a", 5), ("b", 5), ("c", 5)]), [("a", 5), ("b", 5), ("c", 5)])
    _check("already sorted", sort_alerts([("a", 9), ("b", 5), ("c", 1)]),
           [("a", 9), ("b", 5), ("c", 1)])
    _check("ascending input reversed", sort_alerts([("a", 1), ("b", 5), ("c", 9)]),
           [("c", 9), ("b", 5), ("a", 1)])
    _check("smallest at index 0 is moved (FIX 1)",
           insertion_sort_alerts([("a", 1), ("b", 2)]), [("b", 2), ("a", 1)])

    print("--- is_blocked ---")
    fast = build_blocklist(blocklist)
    for q, exp in [("10.0.0.9", True), ("10.0.0.5", True), ("8.8.8.8", False),
                   ("172.16.4.2", True)]:
        _check(f"list  {q}", is_blocked(blocklist, q), exp)
        _check(f"set   {q}", is_blocked(fast, q), exp)
    _check("empty blocklist (list)", is_blocked([], "10.0.0.5"), False)
    _check("empty blocklist (set)", is_blocked(build_blocklist([]), "10.0.0.5"), False)
    _check("last element found", is_blocked(blocklist, "172.16.4.2"), True)
```

```
    print("--- find_signature ---")
    _check("given data", find_signature(log, pattern), [0, 8])
    _check("pattern at very end", find_signature("xxxxFAIL", "FAIL"), [4])
    _check("pattern longer than log", find_signature("FA", "FAIL"), [])
    _check("pattern equals log", find_signature("FAIL", "FAIL"), [0])
    _check("overlapping matches", find_signature("aaaa", "aa"), [0, 1, 2])
    _check("empty log", find_signature("", "FAIL"), [])
    _check("empty pattern", find_signature("FAIL", ""), [])
    _check("no match", find_signature("abcdef", "xyz"), [])
    _check("mismatch on last char is not a hit", find_signature("FAIX", "FAIL"), [])

    print("--- closest_logins ---")
    _check("given data (optimised)", closest_logins(events)[0], (0, 5))
    _check("given distance", round(closest_logins(events)[1], 6), round(math.sqrt(2), 6))
    _check("given data (brute force)", closest_logins_bruteforce(events)[0], (0, 5))
    _check("empty list", closest_logins([]), (None, None))
    _check("one event", closest_logins([(1, 1)]), (None, None))
    _check("two events", closest_logins([(0, 0), (3, 4)]), ((0, 1), 5.0))
    _check("duplicate locations -> distance 0",
           closest_logins([(5, 5), (9, 9), (5, 5)]), ((0, 2), 0.0))
    _check("tie keeps first pair", closest_logins([(0, 0), (1, 0), (2, 0)])[0], (0, 1))
    rnd = random.Random(7)
    for t in range(300):                        # optimised == brute force, many ties
        pts = [(rnd.randint(0, 12), rnd.randint(0, 12)) for _ in range(rnd.randint(2, 40))]
        assert closest_logins(pts) == closest_logins_bruteforce(pts), pts
    print("[PASS] 300 random grids: optimised result identical to brute force")
    print("\nALL TESTS PASSED")

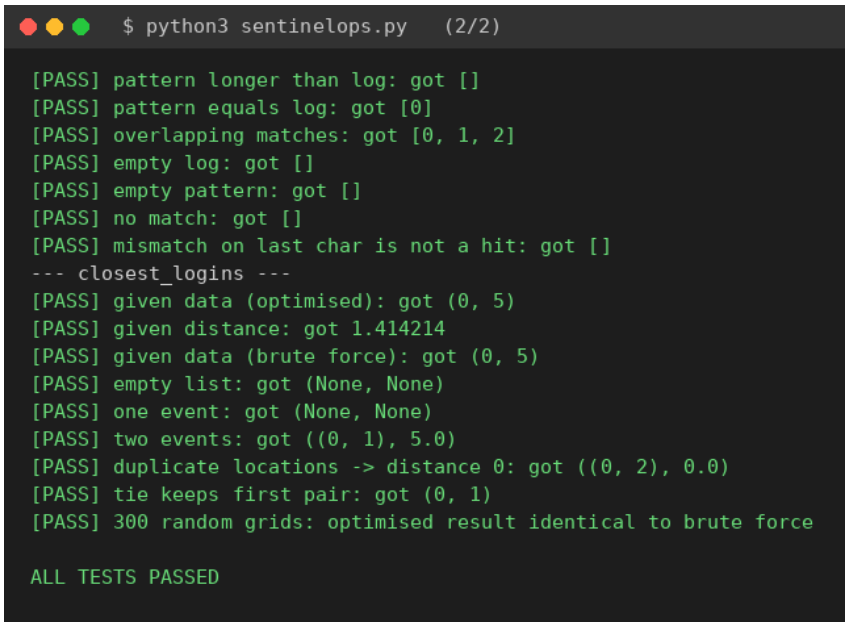

if __name__ == "__main__":
    run_tests()
```

Test-harness execution

```
$ python3 sentinelops.py   (1/2)

--- sort_alerts ---
[PASS] given data (fast): got [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]
[PASS] given data (insertion): got [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]
[PASS] empty list: got []
[PASS] single alert: got [('X', 4)]
[PASS] all equal severity keeps arrival order: got [('a', 5), ('b', 5), ('c', 5)]
[PASS] already sorted: got [('a', 9), ('b', 5), ('c', 1)]
[PASS] ascending input reversed: got [('c', 9), ('b', 5), ('a', 1)]
[PASS] smallest at index 0 is moved (FIX 1): got [('b', 2), ('a', 1)]
--- is_blocked ---
[PASS] list  10.0.0.9: got True
[PASS] set   10.0.0.9: got True
[PASS] list  10.0.0.5: got True
[PASS] set   10.0.0.5: got True
[PASS] list  8.8.8.8: got False
[PASS] set   8.8.8.8: got False
[PASS] list  172.16.4.2: got True
[PASS] set   172.16.4.2: got True
[PASS] empty blocklist (list): got False
[PASS] empty blocklist (set): got False
[PASS] last element found: got True
--- find_signature ---
[PASS] given data: got [0, 8]
[PASS] pattern at very end: got [4]
```

Screenshot 6 – Test harness output (part 1)



Screenshot 7 – Test harness output (part 2): all tests passed

GitHub submission

Repository link: https://github.com/<your-username>/sentinelops-co2 (replace with your actual link before submitting)

Suggested repository contents: original.py, corrected.py, sentinelops.py (final module + test harness), experiments.py, README.md and this report. Share the link through Google Classroom.

Appendix A – Original program (with driver)

```
import math

def sort_alerts(alerts):  # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):  # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):  # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist



if __name__ == "__main__":
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
    log, pattern = "FAILFAIXFAIL", "FAIL"
    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]  # (x, y) in km

    print("sort_alerts      :", sort_alerts(list(alerts)))
    for q in queries:
        print(f"is_blocked({q!r:12}):", is_blocked(blocklist, q))
    print("find_signature   :", find_signature(log, pattern))
    print("closest_logins   :", closest_logins(events))
```

Appendix B – Corrected program (minimal fixes, FIX 1 – FIX 9)

```
import math

def sort_alerts(alerts):  # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
```

```
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:      # FIX 1: j > 0  ->  j >= 0
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key                           # FIX 2: alerts[j]  ->  alerts[j + 1]
    return alerts

def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return True                               # FIX 3: False  ->  True
        # FIX 4: removed the 'else: return False' that quit after the first element
    return False                                      # FIX 4: reached only after ALL entries
were checked

def find_signature(log, pattern):  # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    if m == 0 or m > n:                               # boundary guard: nothing to find
        return []
    hits = []
    for i in range(n - m + 1):                        # FIX 5: n - m  ->  n - m + 1
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:                                         # FIX 6: 'if j == m-1' also fired after a
break
            hits.append(i)
    return hits

def closest_logins(events):  # brute-force closest pair of login locations
    min_dist = float("inf")                           # FIX 7: 0  ->  infinity
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):           # FIX 8: range(len) -> range(i + 1, len)
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < min_dist:                          # FIX 9: '>'  ->  '<'
                min_dist = d
                pair = (i, j)
    if pair is None:                                  # fewer than two events
        return None, None
    return pair, min_dist
```

Appendix C – Experiment script

experiments.py (included in the repository) generates the data of part (d): it counts distance computations for closest_logins at n = 1,000 / 2,000 / 4,000 / 8,000, counts insertion-sort and Timsort comparisons on random and nearly sorted alerts, and times the blocklist lookups.

Code of Conduct

I Deepak S (Reg. No. 192511124) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____